



INGENIERÍA WEB

Mtr. Jisbaj Gamarra Salas

Guía de laboratorio 03

Tema: Fundamentos de HTML y CSS

Ingeniería Web

Propósito de la sesión

- Comprender por qué HTML y CSS siguen estándares.
- Diferenciar etiquetas válidas de comportamientos no estándar.
- Aplicar selectores CSS con mayor precisión.
- Entender herencia, especificidad y colisiones de estilos.
- Introducir el modelo de caja.
- Prepararse para depurar estilos con las herramientas del navegador.

Ruta de trabajo

1. Rutas relativas y absolutas
2. HTML: contexto histórico y estándares
3. Selectores CSS
4. Especificidad e herencia
5. Pseudoclases para enlaces
6. Modelo de caja
7. Depuración con inspector

Rutas en recursos web

Las rutas aparecen con frecuencia en:

- `img src`
- `a href`
- `link href`

Dos formas comunes:

- **Relativa**: parte desde el archivo actual
- **Absoluta**: apunta a una dirección completa

Ejemplos de rutas

```
<a href="acerca.html">Acerca del curso</a>  
  
<link rel="stylesheet" href="css/estilos.css" />  
<a href="https://developer.mozilla.org/">MDN</a>
```

Idea clave:

- Usa rutas relativas para archivos del proyecto.
- Usa rutas absolutas cuando el recurso vive fuera del proyecto.

HTML y los estándares

Pregunta guía:

¿Puedo inventar mis propias etiquetas HTML y esperar que el navegador las entienda correctamente?

Los navegadores suelen hacer un esfuerzo por mostrar el contenido, incluso cuando el HTML no sigue el estándar.

Eso explica por qué, a veces, código inválido **parece** funcionar.

Ejemplo: etiqueta no estándar

En este ejemplo se usa una etiqueta inventada llamada `aviso`.

HTML

```
<p>La práctica se entrega <aviso>el viernes a las 6:00 p. m.</aviso></p>
```

CSS

```
aviso {  
  background: #fff3a3;  
  padding: 4px 10px;  
  border-radius: 8px;  
  font-weight: bold;  
}
```

Resultado esperado del ejemplo

La práctica se entrega **el viernes a las 6:00 p.
m.**

El navegador lo muestra, pero el elemento no es estándar.

¿Por qué se muestra si no es estándar?

Porque los navegadores aplican una lógica de **tolerancia a errores**:

- intentan renderizar aunque el documento no sea perfecto
- evitan "romper" páginas antiguas
- privilegian la visualización del contenido

Pero esto **no convierte** ese HTML en correcto.

¿Por qué importa respetar el estándar?

Porque lo no estándar:

- puede fallar en otro navegador o dispositivo
- puede romperse en versiones futuras
- dificulta mantenimiento, validación y accesibilidad
- genera comportamientos impredecibles

Regla práctica:

Si quieres estilo, usa clases e ids; no inventes etiquetas.

Breve contexto histórico

- **1989**: nace la World Wide Web
- **1994**: se crea el W3C
- **1997**: HTML4 se consolida como versión estable
- Años después aparecen propuestas más estrictas
- Finalmente se adopta una evolución más realista del lenguaje: HTML moderno

Idea histórica importante

La web creció con millones de páginas imperfectas.

Por eso los navegadores no pueden comportarse como un compilador rígido.

En vez de detenerse con error, hacen “lo mejor posible”.

Eso mantiene viva la web, pero también exige que el desarrollador escriba con disciplina.

Qué debes recordar

Para elegir elementos HTML válidos:

- consulta documentación confiable
- prioriza semántica sobre improvisación

Para saber si algo está implementado:

- revisa compatibilidad entre navegadores

Principio general:

que algo funcione hoy no significa que sea una buena práctica.

Selectores CSS: panorama general

Tres selectores base:

- **Elemento** → `p`
- **Id** → `#encabezado`
- **Clase** → `.tarea`

Cada uno apunta a un conjunto distinto de elementos.

Clases e ids

- Usa **id** cuando el elemento sea único en la página.
- Usa **class** cuando varios elementos compartan estilo o rol visual.

Ejemplo típico:

- un título principal con `id`
- varias etiquetas visuales con `class`

Código: clases e ids

HTML

```
<h1 id="encabezado">Pendientes del laboratorio</h1>  
  
<span class="tarea urgente">Subir avance 1</span>  
<span class="tarea">Leer la guía</span>  
<span class="tarea urgente">Corregir estilos</span>
```

CSS

```
#encabezado {  
  color: #17336b;  
}  
  
.tarea {  
  padding: 10px 16px;  
  border-radius: 999px;  
  background: #dbe7ff;  
}  
  
.urgente {  
  background: #ffe2e2;  
  color: #9b1c1c;  
}
```

Resultado esperado del ejemplo

Pendientes del laboratorio

Subir avance 1

Leer la guía

Corregir estilos

Las tareas en rojo requieren atención inmediata.

Selector por elemento y clase

También puedes combinar ambos:

- `p.alerta` → selecciona solo párrafos con la clase `alerta`
- `li.importante` → selecciona solo elementos `li` con esa clase

Esto permite ser más preciso sin usar ids.

Selector descendiente

Sintaxis:

```
contenedor elemento
```

Ejemplo:

```
.tarjeta strong
```

Selecciona los elementos `strong` que estén dentro de `.tarjeta`, aunque estén anidados varios niveles abajo.

Código: selector descendiente

HTML

```
<section class="tarjeta">
  <h2>Ficha del curso</h2>
  <p>En esta unidad aprenderás a usar <strong>selectores CSS</strong>.</p>
  <div class="pie">Estado: <strong>activo</strong></div>
</section>

<p>Fuera de la tarjeta, este <strong>texto</strong> no cambia.</p>
```

CSS

```
<style>
  .tarjeta strong {
    color: #7c3aed;
  }

  .pie strong {
    color: #059669;
  }
```

Resultado esperado del ejemplo

Ficha del curso

En esta unidad aprenderás a usar **selectores CSS** para apuntar a zonas específicas del documento.

Estado: **activo**

Fuera de la tarjeta, este **texto** no hereda el mismo color.

Observación importante

El selector descendiente no exige que el elemento sea hijo directo.

Basta con que esté **dentro** del contenedor.

Esto ayuda a:

- evitar repetir clases innecesarias
- organizar mejor el HTML
- aplicar estilo a grupos completos desde un contenedor

Selector agrupado con coma

Sintaxis:

```
h2, .resumen, footer
```

Se usa cuando varios selectores comparten la misma regla.

Ventaja:

- reduce duplicación
- hace más legible la hoja de estilos

Selectores combinados

Un selector puede crecer en precisión.

Ejemplo:

```
#panel li.destacado strong
```

Se interpreta así, de derecha a izquierda:

- selecciona `strong`
- que esté dentro de un `li`
- cuyo `li` tenga clase `destacado`
- y todo eso dentro de `#panel`

Código: selectores combinados

HTML

```
<div id="panel">
  <ul>
    <li>Revisar estructura HTML</li>
    <li class="destacado">Validar <strong>enlaces internos</strong></li>
    <li class="destacado">Probar <strong>estilos responsivos</strong></li>
  </ul>
</div>
```

CSS

```
#panel li.destacado strong {  
    color: #c2410c;  
}
```

Resultado esperado del ejemplo

Checklist de publicación

- Revisar estructura HTML
- Validar **enlaces internos**
- Probar **estilos responsivos**
- Comprimir imágenes

Errores frecuentes al escribir selectores

Observa la diferencia:

- `p.alerta` → párrafo con clase `alerta`
- `p .alerta` → cualquier descendiente con clase `alerta` dentro de un párrafo

Y también:

- `p .alerta` no es lo mismo que `p, .alerta`

Pequeños espacios cambian mucho el resultado.

Colisión de estilos

Dos reglas pueden apuntar al mismo elemento.

Cuando eso ocurre, CSS decide con base en **especificidad**.

Ejemplo conceptual:

- una regla general para `strong`
- una regla más precisa para `.mensaje strong`

La más específica gana.

Código: especificidad

HTML

```
<div class="mensaje">Prioridad de revisión: <strong>alta</strong></div>
```

CSS

```
strong {  
  color: #2563eb;  
}  
  
.mensaje strong {  
  color: #dc2626;  
}
```

Resultado esperado del ejemplo

Prioridad de revisión: **alta**

La regla `.mensaje strong` gana por ser más específica.

Reglas básicas de especificidad

En términos generales:

- `id` gana a `class`
- `class` gana a selector de elemento
- una regla directa suele ganar frente a una heredada
- si la especificidad empata, gana la regla escrita más abajo

Esto forma parte de la **cascada** de CSS.

Herencia

Algunas propiedades pasan del padre a los hijos.

Ejemplos que suelen heredarse:

- `font-family`
- `color`
- `font-size`

Otras, relacionadas con distribución y caja, normalmente no se heredan.

Herencia y enlaces

Aunque el texto suele heredar color y tipografía, los enlaces tienen estilos iniciales del navegador.

Por eso muchas veces un `a` no se ve como el resto del texto aunque esté dentro del mismo contenedor.

Para controlarlo, debes estilizar `a` explícitamente.

Código: herencia y enlaces

HTML

```
<div class="caja">
  <p>
    Consulta también la
    <a href="#">guía del proyecto final</a>
    y revisa los <em>criterios de evaluación</em>.
  </p>
</div>
```

CSS

```
body {  
  font-family: Georgia, serif;  
  color: #be185d;  
}  
  
a {  
  color: #0f766e;  
  text-decoration: none;  
  font-weight: 700;  
}
```

Resultado esperado del ejemplo

Consulta también la **guía del proyecto final** y revisa los *criterios de evaluación*.

Pseudoclases de enlace

Las pseudoclases representan estados.

Ejemplos útiles:

- `a:link` → enlace no visitado
- `a:visited` → enlace visitado
- `a:hover` → cuando el cursor pasa encima
- `a:active` → cuando el enlace se activa

Sirven para mejorar retroalimentación visual e interacción.

Nota de estilo

Aunque podrías construir toda una página con `div` y `span`, no es lo ideal.

Conviene usar elementos semánticos cuando corresponda:

- `h1`, `p`, `strong`
- `section`, `article`, `nav`, `footer`

Esto mejora comprensión, mantenimiento y accesibilidad.

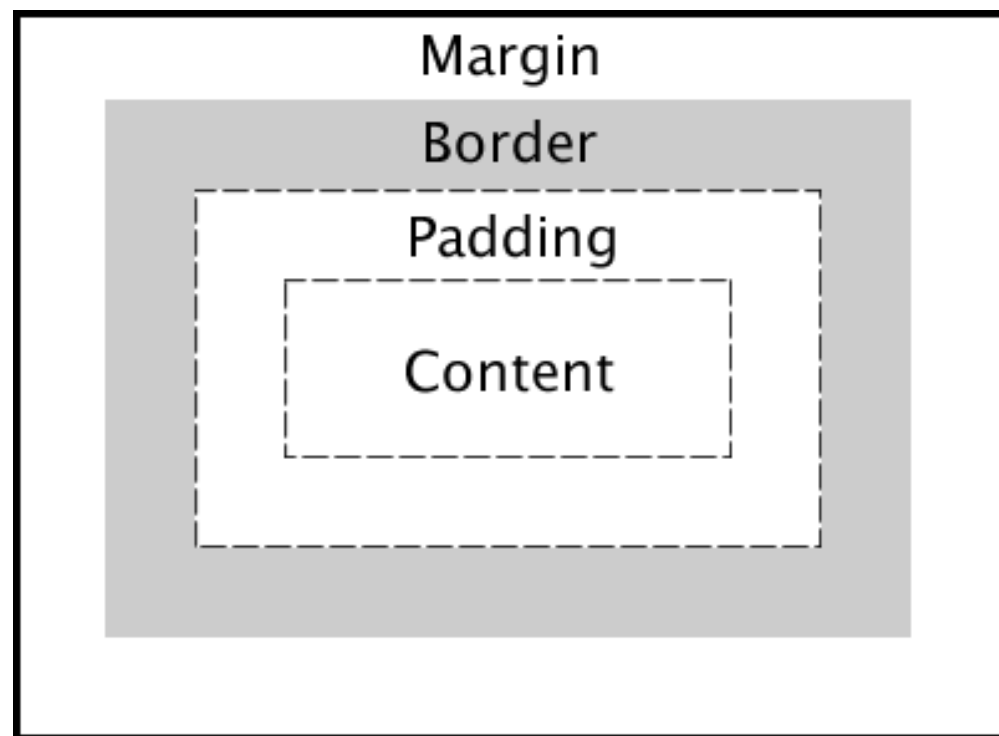
Modelo de caja

Modelo de caja

Todo elemento se entiende como una caja con cuatro zonas:

1. **contenido**
2. **padding**
3. **border**
4. **margin**

Comprender esto es clave para controlar espacio y distribución.



Border

El borde rodea el contenido y el padding.

Se puede definir con la forma abreviada:

```
border: 4px solid #7c3aed;
```

O por lados:

```
border-top  
border-right  
border-bottom  
border-left
```

Código: borde y padding

HTML

```
<div class="etiqueta">Entrega validada</div>
```

CSS

```
.etiqueta {  
  border: 4px solid #7c3aed;  
  padding: 18px 28px;  
  border-radius: 16px;  
}
```

Resultado esperado del ejemplo

Entrega validada

El padding crea espacio interior entre el contenido y el borde.

Padding

El `padding` es el espacio **interior** entre el contenido y el borde.

Ejemplos:

```
padding: 20px;  
padding: 10px 24px;  
padding: 8px 16px 12px 16px;
```

Se lee así:

- un valor → todos los lados
- dos valores → vertical | horizontal
- cuatro valores → arriba | derecha | abajo | izquierda

Margin

El `margin` es el espacio **exterior** entre una caja y otras cajas.

Se usa para separar elementos entre sí.

Si una caja ya se ve bien por dentro pero está demasiado pegada a otra, normalmente necesitas `margin`, no `padding`.

Código: separación con margin

HTML

```
<div class="bloque">Bloque 1: introducción al problema.</div>  
<div class="bloque">Bloque 2: propuesta de solución.</div>  
<div class="bloque">Bloque 3: validación final.</div>
```

CSS

```
<style>
  .bloque {
    border: 2px solid #2563eb;
    padding: 18px 22px;
  }

  .bloque + .bloque {
    margin-top: 18px;
  }
```

Resultado esperado del ejemplo

Bloque 1: introducción al problema.

Bloque 2: propuesta de solución.

Bloque 3: validación final.

Bordes redondeados

`border-radius` permite suavizar esquinas.

No necesitas definir un borde visible para usarlo.

Se usa mucho en:

- tarjetas
- botones
- badges
- avatares
- paneles modernos

Código: border-radius

HTML

```
<div class="perfil">  
  <div class="avatar">LW</div>  
  <div>  
    <div class="nombre">Lucía Wong</div>  
    <div class="cargo">Asistente del laboratorio</div>  
  </div>  
</div>
```

CSS

```
.perfil {  
  border-radius: 28px;  
}  
  
.avatar {  
  width: 92px;  
  height: 92px;  
  border-radius: 50%;  
}
```

Resultado esperado del ejemplo



Lucía Wong

Asistente del laboratorio de Ingeniería Web

Cómo depurar con el inspector

Cuando algo “no sale como esperabas”, abre las herramientas del navegador.

Revisa:

- qué selector está aplicando realmente
- qué regla fue sobrescrita
- qué propiedades están heredadas
- cuánto padding, border y margin tiene el elemento

El inspector convierte el CSS en algo visible y comprobable.

Caso de práctica

Construye una mini ficha de curso que incluya:

- título principal
- lista de pendientes
- una tarjeta con descripción
- un enlace destacado
- separación visual usando margin y padding
- esquinas redondeadas

Objetivo:

usar en una misma página lo aprendido sobre selectores y modelo de caja.

Cierre

Ideas esenciales de la sesión:

- sigue estándares HTML
- usa clases e ids con intención
- domina la diferencia entre selectores parecidos
- recuerda que CSS funciona por cascada
- comprende padding, border y margin
- usa el inspector para verificar en lugar de adivinar

Ejercicio final

Toma uno de los ejemplos de esta guía y modifica:

- colores
- tipografía
- separación interna
- separación entre bloques
- nivel de especificidad

Luego explica qué cambio pertenece a:

- selector
- herencia
- especificidad
- box model



¡Gracias! ➡